

Author: Pankaj Yadav

Date: 04/12/2026

Description:

This assignment is to learn connecting GenAI API and use prompts

Week #4

This week, you'll create a number of prompt-based solutions. Each must be done in Python using an API (most likely OpenAI). It's a good idea to practice using an interface, which is safe for what we're doing because the data isn't real.

Problem #1: Math and Extraction with Zero and One-Shot Prompting

Using the following email, build two prompts: first, create a zero-shot prompt to extract all the information from the email as JSON.

Then, create a one-shot prompt to extract the information as JSON and run the extract on the 2nd email (this will be one prompt with one-shot prompting):

Hey Jim,

I'd like to order some new shoes. Please ship the following:

1. Nike Air Jordan I - 2 pair
2. Converse All-Star - 10 pair
3. New Balance 990 - 1 pair
4. Nike Zoom Fly 5 men's red - 2 pair

Please provide sub-totals and grand total cost.

Thanks.

Lance Gentry 123 Main St. Chelsea, MI 48109 248-229-2229

Extract: shoe_agent_name (this is Jim) request_type (this is "Order") shoe_brand, shoe_model, shoe_quantity customer_name customer_street customer_city customer_state customer_zip customer_phone

In addition, you'll assign a shoe_price for each shoe, calculate the shoe_subtotal for each shoe, and then the grand_total.

Hey Michelle,

I'd like to order some new shoes. Please ship the following:

1. Nike Air Jordan I - 1 pair
2. Converse All-Star - 20 pair
3. New Balance 990 - 2 pair
4. Nike Zoom Fly 5 men's red - 5 pair

Please provide sub-totals and grand total cost.

Thanks.

Artis Gilmore 723 Lexington Blvd. New York, NY 10001 (503) 484-1029

Use one or more prompts to accomplish this extraction. You're resulting JSON should look something like this: Week 4 Exercise Output Example.txt

Problem #2: Chat Completion for Chain of Thought

Re-create the chat from Listing 6.4 in the textbook (shown below), but use OpenAI, not Azure OpenAI.

Q: There were nine computers in the server room. Five more computers were installed each day, from Monday to Thursday. How many computers are now in the server room?

A: There are 4 days from Monday to Thursday. 5 computers were added each day. That means in total $4 * 5 = 20$ computers were added. There were 9 computers initially, so now there are $9 + 20 = 29$ computers. The answer is 29.

Q: Michael had 58 golf balls. On Tuesday, he lost 23 golf balls. On Wednesday, he lost 2 more. How many golf balls did he have at the end of Wednesday?

A: Michael initially had 58 balls. He lost 23 on Tuesday, so after that he has $58 - 23 = 35$ balls. On Wednesday he lost 2 more so now he has $35 - 2 = 33$ balls. The answer is 33.

Q: Olivia has 23. *She bought five bagels for 3 each.* How much money does she have left?

A: She bought 5 bagels for *3 each. This means she spent 15.* She has \$8 left.

Q: When I was 6 my sister was half my age. Now I'm 70 how old is my sister?

Build the solution using a Jupyter Notebook (no other Python is acceptable) and include Markdown that thoroughly explains your thought process and your commentary on the results you achieved. Obvious Markdown that explicitly only explains the code will be ignored, since I can read the code. I'm looking for your commentary and evaluation in your own words. In addition, your Markdown should explicitly compare the similarities and differences between the replies to each of the questions and each of the responses for Problem #1.

Explanation

First step was to install the OpenAI python library. Once installed I created a project in OpenAI's API platform and created a API_key. Put some balance in there.

I setup Json, os and openAI libraries. Json to convert the results from openAI , os.getenv to fetch the APIkey from the terminal path instead of using the key in code itself, and OpenAI to connect and communicate with model.

```
In [7]: import json
import os
from openai import OpenAI
api_key = os.getenv("OPENAI_API_KEY")
```

Lets define a function based on OpenAI's API documentation (<https://developers.openai.com/api/docs/guides/prompt-engineering?lang=python>) for responses. I will use this function to fetch the results and print them. this will make the code little sleek.

```
In [8]: def openai_prompt(prompt, effort="low"):
        client = OpenAI(api_key=api_key)
        result = client.responses.create(
            model="gpt-5.1",
            reasoning={ "effort": effort },
            input=(prompt),
        )
        return result.output_text
```

In this first prompt I am creating the Zero shot prompt which will be just my instructions in the prompt and the email. While writing the prompt I made sure that everything is provided to avoid any hallucinations that model could face. The structure I chose to provide the what to do first, then what should be the output and the specific rules regarding the JSON outputs. This specific step is to avoid hallucinations. At the end, the email is provided which is the input.

```
In [9]: # Lets define our Zero prompt for the shoe order email.

prompt = """
Extract all required information from the email below and output ONLY valid JSON. Do not include explanations,
notes, or commentary. In addition, you'll assign a shoe_price for each shoe, calculate the shoe_subtotal for each shoe,
and then the grand_total.

Extract the following fields:

- shoe_agent_name (the person being addressed in the greeting)
```

- request_type (always "Order")
- customer_name
- customer_street
- customer_city
- customer_state
- customer_zip
- customer_phone
- items: an array where each item contains:
 - shoe_brand
 - shoe_model
 - shoe_quantity
 - shoe_price (assign a reasonable price yourself)
 - shoe_subtotal (quantity × price)
- grand_total (sum of all shoe_subtotals)

Rules:

- Output must be valid JSON.
- shoe_price must be a number.
- shoe_subtotal and grand_total must be correct mathematical calculations.
- Preserve the exact text of shoe brand and model as written in the email.

Email:

Hey Jim,

I'd like to order some new shoes. Please ship the following:

1. Nike Air Jordan I – 2 pair
2. Converse All-Star – 10 pair
3. New Balance 990 – 1 pair
4. Nike Zoom Fly 5 men's red – 2 pair

Please provide sub-totals and grand total cost.

Thanks.

Lance Gentry
123 Main St.
Chelsea, MI 48109
248-229-2229

```
results = json.loads(openai_prompt(prompt))
print(json.dumps(results, indent=2))
```

```
{
  "shoe_agent_name": "Jim",
  "request_type": "Order",
  "customer_name": "Lance Gentry",
  "customer_street": "123 Main St.",
  "customer_city": "Chelsea",
  "customer_state": "MI",
  "customer_zip": "48109",
  "customer_phone": "248-229-2229",
  "items": [
    {
      "shoe_brand": "Nike",
      "shoe_model": "Air Jordan I",
      "shoe_quantity": 2,
      "shoe_price": 180,
      "shoe_subtotal": 360
    },
    {
      "shoe_brand": "Converse",
      "shoe_model": "All-Star",
      "shoe_quantity": 10,
      "shoe_price": 60,
      "shoe_subtotal": 600
    },
    {
      "shoe_brand": "New Balance",
      "shoe_model": "990",
      "shoe_quantity": 1,
      "shoe_price": 200,
      "shoe_subtotal": 200
    },
    {
      "shoe_brand": "Nike",
      "shoe_model": "Zoom Fly 5 men's red",
      "shoe_quantity": 2,
      "shoe_price": 160,
      "shoe_subtotal": 320
    }
  ],
  "grand_total": 1480
}
```

As you see above the zero-shot extraction produced valid structured JSON and successfully captured the core entities:

- customer identity and address fields
- item-level attributes
- computed subtotals and grand total

As next prompt will be a one shot prompt and I am adding the output from first prompt as the input to the second prompt. So this prompt will have my `prompt` + `the email` + `the JSON output from last prompt` + `the new email`. Now the use of example will already set a base for the GPT model.

```
In [10]: prompt = """
Extract all required information from the email below and output ONLY valid JSON. Do not include explanations,
notes, or commentary. In addition, you'll assign a shoe_price for each shoe, calculate the shoe_subtotal for each shoe,
and then the grand_total. Use exactly the same keys and data types as the example JSON output below.

Extract the following fields:

- shoe_agent_name (the person being addressed in the greeting)
- request_type (always "Order")
- customer_name
- customer_street
- customer_city
- customer_state
- customer_zip
- customer_phone
- items: an array where each item contains:
    - shoe_brand
    - shoe_model
    - shoe_quantity
    - shoe_price (assign a reasonable price yourself)
    - shoe_subtotal (quantity × price)
- grand_total (sum of all shoe_subtotals)

Here is an example of the email and expected JSON output format extracted from it:

Example Email:
Hey Jim,

I'd like to order some new shoes. Please ship the following:

1. Nike Air Jordan I - 2 pair
2. Converse All-Star - 10 pair
```

3. New Balance 990 – 1 pair
4. Nike Zoom Fly 5 men's red – 2 pair

Please provide sub-totals and grand total cost.

Thanks.

Lance Gentry
123 Main St.
Chelsea, MI 48109
248-229-2229

Example of JSON Output:

```
{
  "shoe_agent_name": "Jim",
  "request_type": "Order",
  "customer_name": "Lance Gentry",
  "customer_street": "123 Main St.",
  "customer_city": "Chelsea",
  "customer_state": "MI",
  "customer_zip": "48109",
  "customer_phone": "248-229-2229",
  "items": [
    {
      "shoe_brand": "Nike",
      "shoe_model": "Air Jordan I",
      "shoe_quantity": 2,
      "shoe_price": 180,
      "shoe_subtotal": 360
    },
    {
      "shoe_brand": "Converse",
      "shoe_model": "All-Star",
      "shoe_quantity": 10,
      "shoe_price": 60,
      "shoe_subtotal": 600
    },
    {
      "shoe_brand": "New Balance",
      "shoe_model": "990",
      "shoe_quantity": 1,
      "shoe_price": 200,
      "shoe_subtotal": 200
    },
  ],
}
```

```
{
  "shoe_brand": "Nike",
  "shoe_model": "Zoom Fly 5 men's red",
  "shoe_quantity": 2,
  "shoe_price": 170,
  "shoe_subtotal": 340
},
"grand_total": 1500
}
```

Rules:

- Output must be valid JSON.
- Return only one valid JSON object, no markdown, no code fences.
- shoe_price must be a number and should be same if the brand and model are the same in input JSON.
- shoe_subtotal and grand_total must be correct mathematical calculations.
- Preserve the exact text of shoe brand and model as written in the email.
- If a value is missing, use null for that value in the JSON.

Here is the email to extract information from:

Hey Michelle,

I'd like to order some new shoes. Please ship the following:

1. Nike Air Jordan I - 1 pair
2. Converse All-Star - 20 pair
3. New Balance 990 - 2 pair
4. Nike Zoom Fly 5 men's red - 5 pair

Please provide sub-totals and grand total cost.

Thanks.

Artis Gilmore
723 Lexington Blvd.
New York, NY 10001
(503) 484-1029

```
results = json.loads(openai_prompt(prompt))
print(json.dumps(results, indent=2))
```



```

{
  "shoe_agent_name": "Michelle",
  "request_type": "Order",
  "customer_name": "Artis Gilmore",
  "customer_street": "723 Lexington Blvd.",
  "customer_city": "New York",
  "customer_state": "NY",
  "customer_zip": "10001",
  "customer_phone": "(503) 484-1029",
  "items": [
    {
      "shoe_brand": "Nike",
      "shoe_model": "Air Jordan I",
      "shoe_quantity": 1,
      "shoe_price": 180,
      "shoe_subtotal": 180
    },
    {
      "shoe_brand": "Converse",
      "shoe_model": "All-Star",
      "shoe_quantity": 20,
      "shoe_price": 60,
      "shoe_subtotal": 1200
    },
    {
      "shoe_brand": "New Balance",
      "shoe_model": "990",
      "shoe_quantity": 2,
      "shoe_price": 200,
      "shoe_subtotal": 400
    },
    {
      "shoe_brand": "Nike",
      "shoe_model": "Zoom Fly 5 men's red",
      "shoe_quantity": 5,
      "shoe_price": 170,
      "shoe_subtotal": 850
    }
  ],
  "grand_total": 2630
}

```

Even though both the outputs are same structurally, one-shot extraction have stronger chances in consistency because the model had a concrete target schema to imitate. In my results, one-shot prompting supposed to have a:

- output formatting stability
- adherence to exact field names
- consistency in pricing logic across repeated item brands/models
- lower risk of missing keys

A key observation is that the model can perform both extraction and arithmetic in one pass, but arithmetic reliability depends on clear constraints. Including explicit rules for numeric fields and totals materially improved output quality.

The model correctly, calculated the final totals for both emails. the subtotals were correct as well. Captured the correct customer demographic information.

In my results, both zero-shot and one-shot were correct, but one-shot was more dependable as a repeatable extraction strategy. The example acted as a template, which reduced variance and made the output format more stable.

Problem #2: Chat Completion for Chain of Thought

```
In [11]: print("AI Response: ",openai_prompt("""
Q: There were nine computers in the server room. Five more computers were installed each day, from Monday to Thursday.
A: There are 4 days from Monday to Thursday. 5 computers were added each day. That means in total  $4 * 5 = 20$  computers
Q: Michael had 58 golf balls. On Tuesday, he lost 23 golf balls. On Wednesday, he lost 2 more. How many golf balls did
A: Michael initially had 58 balls. He lost 23 on Tuesday, so after that he has  $58 - 23 = 35$  balls. On Wednesday he lost
Q: Olivia has $23. She bought five bagels for $3 each. How much money does she have left?
A: She bought 5 bagels for $3 each. This means she spent $15. She has $8 left.
Q: When I was 6 my sister was half my age. Now I'm 70 how old is my sister?
"""),sep="\n")
```

AI Response:

When you were 6, your sister was half your age, so she was 3.

The age difference is $6 - 3 = 3$ years and that never changes.

Now you're 70, so your sister is $70 - 3 = 67$ years old.

The CoT style examples guided the model to follow a step by step pattern for arithmetic word problems. The responses were generally direct, numerically correct and formatted as short reasoning + final answer Compared to free-form answering, few-shot Q/A context made the model's style more consistent and reduced the behavior where model just provides the final answer.